

UNIT II

Divide and Conquer Method: General Method – binary search – maximum and minimum – merge sort - quick sort – Strassen's Matrix multiplication. Greedy Method: General method – knapsack problem – minimum spanning tree algorithms – single source shortest path algorithm – scheduling, optimal storage on tapes, optimal merge patterns.

2 Marks

1. Define the divide and conquer method. (UQ APRIL'13 & APRIL'12, Nov'16)

Given a function to compute on 'n' inputs the divide-and-conquer strategy suggests splitting the inputs including 'k' sub problems. The sub problems must be solved, and then a method must be found to combine sub solutions into a solution of the whole. If the sub problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied.

2. Define control abstraction.

A control abstraction we mean a procedure whose flow of control is clear but whose primary operations are by other procedures whose precise meanings are left undefined.

3. Write the Control abstraction for Divide-and conquer.

```
Algorithm DandC (k)
{
  If small(p) then return(s)
  else
  {
    Apply D and C to each of these subproblems
    Return combine (DAnd C(_1) DAnd C(_2),----, DAnd ((_k));
  }
}
```

4. What is the substitution method?

One of the methods for solving any such recurrence relation is called the substitution method.

5. What is the binary search? (UQ APRIL'12/APRIL 2015, Nov'16)

If 'q' is always chosen such that 'aq' is the middle element (that is, $q = \lfloor (n+1)/2 \rfloor$), then the resulting search algorithm is known as binary search.

6. Give computing time for Binary search?

In conclusion we are now able completely describe the computing time of binary search by giving formulas that describe the best, average and worst cases.

Successful searches

$O(1)$ $O(\log n)$ $O(\log n)$
best average worst

Unsuccessful searches

$O(\log n)$
best, average, worst

7. What is the maximum and minimum problem?

The problem is to find the maximum and minimum items in a set of 'n' elements. Though this problem may look so simple as to be contrived, it allows us to demonstrate divide and- conquer in simple setting.

8. What is the Quick sort? (UQ Nov'10& APRIL'12)

The n numbers in quick sort, the division into sub arrays is made so that the sorted sub arrays do not need to be merged later.

9. Write the Analysis for the Quick sot.

In analyzing QUICKSORT, we can only make the number of element comparisons $c(n)$. It is easy to see that the frequency count of other operations is of the same order as $C(n)$.

10. Is insertion sort better than the merge sort? (UQ APRIL'13)

Insertion sort works exceedingly fast on arrays of less than 16 elements, though for large 'n' its computing time is $O(n^2)$.

11. Write a algorithm for straightforward maximum and minimum>

```
Algorithm straight MaxMin(a,n,max,min)
//set max to the maximum and min to the minimum of a[1:n]
{
max := min: = a[i];
for i = 2 to n do
{
if(a[i] >max) then max: = a[i];
if(a[i] >min) then min: = a[i];
}
}
```

12. Give the recurrence relation of divide-and-conquer?

The recurrence relation is

$$T(n) = g(n)$$

$$T(n_1) + T(n_2) + \dots + T(n_k) + f(n)$$

13. Write the algorithm for Iterative binary search?

```
Algorithm BinSearch(a,n,x)
//Given an array a[1:n] of elements in nondecreasing
// order,  $n > 0$ , determine whether  $x$  is present
{
low : = 1;
high : = n;
while (low < high) do
{
mid : =  $\lceil (low+high)/2 \rceil$ ;
then high:= mid-1;
else return mid;
}
return 0;
}
```

14. What is meant by feasible solution?

Given n inputs and we are required to form a subset such that it satisfies some given constraints then such a subset is called feasible solution.

15. Write any two characteristics of Greedy Algorithm?

- * To solve a problem in an optimal way construct the solution from given set of candidates.
- * As the algorithm proceeds, two other sets get accumulated among this one set contains the candidates that have been already considered and chosen while the other set contains the candidates that have been considered but rejected.

16. Define optimal solution?

A feasible solution either maximizes or minimizes the given objective function is called as optimal solution

17. What is Knapsack problem? (UQ Nov'12)

A bag or sack is given capacity n and n objects are given. Each object has weight w_i and profit p_i . Fraction of object is considered as x_i (i.e) $0 \leq x_i \leq 1$. If fraction is 1 then entire object is put into sack. When we place this fraction into the sack we get $w_i x_i$ and $p_i x_i$.

18. What is the Greedy choice property?

- * The first component is greedy choice property (i.e.) a globally optimal solution can arrive at by making a locally optimal choice.
- * The choice made by greedy algorithm depends on choices made so far but it cannot depend on any future choices or on solution to the sub problem.
- * It progresses in top down fashion.

19. What is greedy method? (UQ:Apr/May'14)

Greedy method is the most important design technique, which makes a choice that looks best at that moment. A given 'n' inputs are required us to obtain a subset that satisfies some constraints that is the feasible solution. A greedy method suggests that one can device an algorithm that works in stages considering one input at a time.

20. What are the steps required to develop a greedy algorithm?

- * Determine the optimal substructure of the problem.
 - * Develop a recursive solution.
 - * Prove that at any stage of recursion one of the optimal choices is greedy choice.
- Thus it is always safe to make greedy choice.
- * Show that all but one of the sub problems induced by having made the greedy choice are empty.
 - * Develop a recursive algorithm and convert into iterative algorithm.

21. What is activity selection problem?

The 'n' task will be given with starting time s_i and finishing time f_i . Feasible Solution is that the task should not overlap and optimal solution is that the task should be completed in minimum number of machine set.

22. What does Job sequencing with deadlines mean?

- * Given a set of 'n' jobs each job 'i' has a deadline d_i such that $d_i \geq 0$ and a profit p_i such that $p_i \geq 0$.
- * For job 'i' profit p_i is earned iff it is completed within deadline.
- * Processing the job on the machine is for 1 unit of time. Only one machine is available.

23. When is a task said to be late in a schedule?

A task is said to be late in any schedule if it finishes after its deadline else a task is early in a schedule.

24. Write the general algorithm for Greedy method control abstraction.

Algorithm Greedy (a, n)

```

{
solution=0;
for i=1 to n do
{
x= select(a);
if feasible(solution ,x) then
solution=Union(solution ,x);
}
return solution;
}

```

25. Define optimal solution for Job sequencing with deadlines.

Feasible solution with maximum profit is optimal solution for Job sequencing with deadlines.

26. Define minimum spanning tree (UQ Nov'12)

Let $G(V,E)$ be an undirected connected graph with vertices 'v' and edge 'E'. A sub-graph $t=(V,E')$ of the G is a Spanning tree of G iff 't' is a tree. 3. The problem is to generate a graph $G'=(V,E)$ where 'E' is the subset of E , G' is a Minimum spanning tree.

27. Write about optimal merge pattern problem. (APRIL 2015)

Two files x_1 and x_2 containing m & n records could be merged together to obtain one merged file. When more than 2 files are to be merged together. The merge can be accomplished by repeatedly merging the files in pairs. An optimal merge pattern tells which pair of files should be merged at each step. An optimal sequence is a least cost sequence.

1. Explain divide and conquer general method

General method:

- Given a function to compute on 'n' inputs the divide-and-conquer strategy suggests splitting the inputs into 'k' distinct subsets, $1 < k \leq n$, yielding 'k' sub problems.
- These sub problems must be solved, and then a method must be found to combine sub solutions into a solution of the whole.
- If the sub problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied.
- Often the sub problems resulting from a divide-and-conquer design are of the same type as the original problem.
- For those cases the re application of the divide-and-conquer principle is naturally expressed by a recursive algorithm.
- D And C(Algorithm) is initially invoked as D and C(P), where 'p' is the problem to be solved.
- Small(P) is a Boolean-valued function that determines whether the i/p size is small enough that the answer can be computed without splitting.
- If this so, the function 'S' is invoked.
- Otherwise, the problem P is divided into smaller sub problems.
- These sub problems P1, P2 ...Pk are solved by recursive application of D And C.
- Combine is a function that determines the solution to p using the solutions to the 'k' sub problems.
- If the size of 'p' is n and the sizes of the 'k' sub problems are n1, n2nk, respectively, then the computing time of D And C is described by the recurrence relation.

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n); & \text{otherwise.} \end{cases}$$

Where $T(n) \rightarrow$ is the time for D And C on any I/p of size 'n'.

$g(n) \rightarrow$ is the time of compute the answer directly for small I/ps.

$f(n) \rightarrow$ is the time for dividing P & combining the solution to sub problems.

1. Algorithm D And C(P)
2. {
3. if small(P) then return S(P);
4. else
5. {
6. divide P into smaller instances

P1, P2... Pk, k>=1;

7. Apply D And C to each of these sub problems;
8. return combine (D And C(P1), D And C(P2),.....,D And C(Pk));
9. }
10. }

- The complexity of many divide-and-conquer algorithms is given by recurrences of the form

$$T(n) = \begin{cases} T(1) & n=1 \\ aT(n/b)+f(n) & n>1 \end{cases}$$

→ Where a & b are known constants.

→ We assume that T(1) is known & 'n' is a power of b(i.e., n=b^k)

- One of the methods for solving any such recurrence relation is called the substitution method.
- This method repeatedly makes substitution for each occurrence of the function. T is the Right-hand side until all such occurrences disappear.

Example:

- 1) Consider the case in which a=2 and b=2. Let T(1)=2 & f(n)=n.

We have,

$$\begin{aligned} T(n) &= 2T(n/2)+n \\ &= 2[2T(n/2/2)+n/2]+n \\ &= [4T(n/4)+n]+n \\ &= 4T(n/4)+2n \\ &= 4[2T(n/4/2)+n/4]+2n \\ &= 4[2T(n/8)+n/4]+2n \\ &= 8T(n/8)+n+2n \\ &= 8T(n/8)+3n \\ &\quad * \\ &\quad * \\ &\quad * \end{aligned}$$

- In general, we see that $T(n)=2^i T(n/2^i)+in$, for any $\log n \geq i \geq 1$.

→ $T(n)=2^{\log n} T(n/2^{\log n}) + n \log n$

→ Corresponding to the choice of $i=\log n$

→ Thus, $T(n)=2^{\log n} T(n/2^{\log n}) + n \log n$

$$\begin{aligned} &= n. T(n/n) + n \log n \\ &= n. T(1) + n \log n \quad [\text{since, } \log 1=0, 2^0=1] \\ &= 2n + n \log n \end{aligned}$$

2. Write algorithm and explain binary search (UQ APRIL'13 & APRIL'12/ APRIL 2015)

1. Algorithm Bin search(a,n,x)
2. // Given an array a[1:n] of elements in non-decreasing
3. //order, n>=0,determine whether 'x' is present and
4. // if so, return 'j' such that x=a[j]; else return 0.
5. {
6. low:=1; high:=n;

```

7. while (low<=high) do
8. {
9.     mid:=[(low+high)/2];
10.    if (x<a[mid]) then high;
11.    else if(x>a[mid]) then
12.        low=mid+1;
13.    else return mid;
14. }
15. return 0;
16. }

```

- Algorithm, describes this binary search method, where Binsrch has 4I/ps a[], I, l & x.
- It is initially invoked as Binsrch (a,1,n,x)
- A non-recursive version of Binsrch is given below.
- This Binsearch has 3 i/ps a,n, & x.
- The while loop continues processing as long as there are more elements left to check.
- At the conclusion of the procedure 0 is returned if x is not present, or 'j' is returned, such that a[j]=x.
- We observe that low & high are integer Variables such that each time through the loop either x is found or low is increased by at least one or high is decreased at least one.
- Thus we have 2 sequences of integers approaching each other and eventually low becomes > than high & causes termination in a finite no. of steps if 'x' is not present.

Example:

- 1) Let us select the 14 entries.
-15,-6,0,7,9,23,54,82,101,112,125,131,142,151.

→ Place them in a[1:14], and simulate the steps Binsearch goes through as it searches for different values of 'x'.

→ Only the variables, low, high & mid need to be traced as we simulate the algorithm.

→ We try the following values for x: 151, -14 and 9.
for 2 successful searches & 1 unsuccessful search.

- Table. Shows the traces of Bin search on these 3 steps.

X=151	low	high	mid
	1	14	7
	8	14	11
	12	14	13
	14	14	14
			Found

x=-14	low	high	mid
	1	14	7
	1	6	3
	1	2	1
	2	2	2
	2	1	Not found

x=9	low	high	mid
	1	14	7
	1	6	3
	4	6	5
			Found

Theorem: Algorithm Binsearch(a,n,x) works correctly.

Proof:

We assume that all statements work as expected and that comparisons such as $x > a[mid]$ are appropriately carried out.

- Initially low =1, high= n,n>=0, and $a[1] \leq a[2] \leq \dots \leq a[n]$.
- If n=0, the while loop is not entered and is returned.
- Otherwise we observe that each time thro' the loop the possible elements to be checked of or equality with x and $a[low], a[low+1], \dots, a[mid], \dots, a[high]$.
- If $x = a[mid]$, then the algorithm terminates successfully.
- Otherwise, the range is narrowed by either increasing low to (mid+1) or decreasing high to (mid-1).
- Clearly, this narrowing of the range does not affect the outcome of the search.
- If low becomes > than high, then 'x' is not present & hence the loop is exited.

3. Explain Maximum and Minimum (UQ APRIL'13)

- Let us consider another simple problem that can be solved by the divide-and-conquer technique.
- The problem is to find the maximum and minimum items in a set of 'n' elements.
- In analyzing the time complexity of this algorithm, we once again concentrate on the no. of element comparisons.
- More importantly, when the elements in $a[1:n]$ are polynomials, vectors, very large numbers, or strings of character, the cost of an element comparison is much higher than the cost of the other operations.
- Hence, the time is determined mainly by the total cost of the element comparison.

```

1. Algorithm straight MaxMin(a,n,max,min)
2. // set max to the maximum & min to the minimum of a[1:n]
3. {
4.     max:=min:=a[1];
5.     for I:=2 to n do
6.     {
7.         if(a[I]>max) then max:=a[I];
8.         if(a[I]<min) then min:=a[I];
9.     }
10. }
```

Algorithm: Straight forward Maximum & Minimum

- Straight MaxMin requires $2(n-1)$ element comparison in the best, average & worst cases.

- An immediate improvement is possible by realizing that the comparison $a[I] < \min$ is necessary only when $a[I] > \max$ is false.
- Hence we can replace the contents of the for loop by,
 If($a[I] > \max$) then $\max := a[I]$;
 Else if ($a[I] < \min$) then $\min := a[I]$;
- Now the best case occurs when the elements are in increasing order.
 → The no. of element comparison is $(n-1)$.
- The worst case occurs when the elements are in decreasing order.
 → The no. of elements comparison is $2(n-1)$
- The average no. of element comparison is $<$ than $2(n-1)$
- On the average $a[I]$ is $>$ than $\max$ half the time, and so, the avg. no. of comparison is $3n/2 - 1$.
- A divide- and conquer algorithm for this problem would proceed as follows:
 → Let $P = (n, a[1], \dots, a[j])$ denote an arbitrary instance of the problem.
 → Here 'n' is the no. of elements in the list $(a[1], \dots, a[j])$ and we are interested in finding the maximum and minimum of the list.
- If the list has more than 2 elements, P has to be divided into smaller instances.
- For example , we might divide 'P' into the 2 instances, $P1 = ([n/2], a[1], \dots, a[n/2])$ & $P2 = (n - [n/2], a[[n/2] + 1], \dots, a[n])$
- After having divided 'P' into 2 smaller sub problems, we can solve them by recursively invoking the same divide-and-conquer algorithm.

Algorithm: Recursively Finding the Maximum & Minimum

1. **Algorithm MaxMin (I,j,max,min)**
2. //a[1:n] is a global array, parameters I & j
3. //are integers, $1 \leq I \leq j \leq n$. The effect is to
4. //set max & min to the largest & smallest value
5. //in a[I:j], respectively.
6. {
7. if($I=j$) then $\max := \min := a[I]$;
8. else if ($I=j-1$) then // Another case of small(p)
9. {
10. if ($a[I] < a[j]$) then
11. {
12. $\max := a[j]$;
13. $\min := a[I]$;
14. }
15. else
16. {
17. $\max := a[I]$;

```

18. min:=a[j];
19. }
20. }
21. else
22. {
23. // if P is not small, divide P into subproblems.
24. // find where to split the set mid:=(I+j)/2;
25. //solve the subproblems
26. MaxMin(I,mid,max.min);
27. MaxMin(mid+1,j,max1,min1);
28. //combine the solution
29. if (max<max1) then max=max1;
30. if(min>min1) then min = min1;
31. }
32. }

```

- The procedure is initially invoked by the statement, MaxMin(1,n,x,y)
- Suppose we simulate MaxMin on the following 9 elements

A: [1] [2] [3] [4] [5] [6] [7] [8] [9]
 22 13 -5 -8 15 60 17 31 47

- A good way of keeping track of recursive calls is to build a tree by adding a node each time a new call is made.
- For this Algorithm, each node has 4 items of information: I, j, max & imin.
- Examining fig: we see that the root node contains 1 & 9 as the values of I & j corresponding to the initial call to MaxMin.
- This execution produces 2 new calls to MaxMin, where I & j have the values 1, 5 & 6, 9 respectively & thus split the set into 2 subsets of approximately the same size.
- From the tree, we can immediately see the maximum depth of recursion is 4. (including the 1st call)
- The include no.s in the upper left corner of each node represent the order in which max & min are assigned values.

No. of element Comparison:

- If T(n) represents this no., then the resulting recurrence relations is

$$T(n) = \begin{cases} T([n/2]) + T([n/2]) + 2 & n > 2 \\ 1 & n = 2 \\ 0 & n = 1 \end{cases}$$

→ When 'n' is a power of 2, $n = 2^k$ for some +ve integer 'k', then

$$\begin{aligned}
 T(n) &= 2T(n/2) + 2 \\
 &= 2(2T(n/4) + 2) + 2 \\
 &= 4T(n/4) + 4 + 2 \\
 &\quad * \\
 &\quad * \\
 &= 2^{k-1}T(2) + \\
 &= 2^{k-1} + 2^{k-2} \\
 &= 2^{k/2} + 2^{k-2} \\
 &= n/2 + n - 2 \\
 &= (n + 2n)/2 - 2
 \end{aligned}$$

$$T(n)=(3n/2)-2$$

*Note that $(3n/3)-3$ is the best-average, and worst-case no. of comparisons when 'n' is a power of 2.

4. Explain merge sort (UQ APRIL '12, APRIL/MAY '14)

- As another example divide-and-conquer, we investigate a sorting algorithm that has the nice property that is the worst case its complexity is $O(n \log n)$
- This algorithm is called merge sort
- We assume throughout that the elements are to be sorted in non-decreasing order.
- Given a sequence of 'n' elements $a[1], \dots, a[n]$ the general idea is to imagine then split into 2 sets $a[1], \dots, a[n/2]$ and $a[[n/2]+1], \dots, a[n]$.
- Each set is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of 'n' elements.
- Thus, we have another ideal example of the divide-and-conquer strategy in which the splitting is into 2 equal-sized sets & the combining operation is the merging of 2 sorted sets into one.

Algorithm For Merge Sort:

1. Algorithm MergeSort(low,high)
2. //a[low:high] is a global array to be sorted
3. //Small(P) is true if there is only one element
4. //to sort. In this case the list is already sorted.
5. {
6. if (low<high) then //if there are more than one element
7. {
8. //Divide P into subproblems
9. //find where to split the set
10. mid = [(low+high)/2];
11. //solve the subproblems.
12. mergesort (low,mid);
13. mergesort(mid+1,high);
14. //combine the solutions .
15. merge(low,mid,high);
16. }
17. }

Algorithm: Merging 2 sorted subarrays using auxiliary storage.

1. Algorithm merge(low,mid,high)
2. //a[low:high] is a global array containing
3. //two sorted subsets in a[low:mid]
4. //and in a[mid+1:high].The goal is to merge these 2 sets into
5. //a single set residing in a[low:high].b[] is an auxiliary global array.
6. {
7. h=low; I=low; j=mid+1;
8. while ((h<=mid) and (j<=high)) do
9. {
10. if (a[h]<=a[j]) then
11. {
12. b[I]=a[h];

```

13.  h = h+1;
14. }
15. else
16. {
17.  b[I]= a[j];
18.  j=j+1;
19. }
20. I=I+1;
21. }
22. if (h>mid) then
23.  for k=j to high do
24.  {
25.    b[I]=a[k];
26.    I=I+1;
27.  }
28. else
29.  for k=h to mid do
30.  {
31.    b[I]=a[k];
32.    I=I+1;
33.  }
34.  for k=low to high do a[k] = b[k];
35. }

```

- Consider the array of 10 elements $a[1:10] = (310, 285, 179, 652, 351, 423, 861, 254, 450, 520)$
- Algorithm Mergesort begins by splitting $a[]$ into 2 sub arrays each of size five ($a[1:5]$ and $a[6:10]$).
- The elements in $a[1:5]$ are then split into 2 sub arrays of size 3 ($a[1:3]$) and 2 ($a[4:5]$)
- Then the items in $a[1:3]$ are split into sub arrays of size 2 $a[1:2]$ & one($a[3:3]$)
- The 2 values in $a[1:2]$ are split to find time into one-element sub arrays, and now the merging begins.

(310| 285| 179| 652, 351| 423, 861, 254, 450, 520)

→ Where vertical bars indicate the boundaries of sub arrays.

→ Elements $a[1]$ and $a[2]$ are merged to yield,
(285, 310|179|652, 351| 423, 861, 254, 450, 520)

→ Then $a[3]$ is merged with $a[1:2]$ and
(179, 285, 310| 652, 351| 423, 861, 254, 450, 520)

→ Next, elements $a[4]$ & $a[5]$ are merged.
(179, 285, 310| 351, 652 | 423, 861, 254, 450, 520)

→ And then $a[1:3]$ & $a[4:5]$
(179, 285, 310, 351, 652| 423, 861, 254, 450, 520)

→ Repeated recursive calls are invoked producing the following sub arrays.
(179, 285, 310, 351, 652| 423| 861| 254| 450, 520)

→ Elements $a[6]$ & $a[7]$ are merged.

→ Then a[8] is merged with a[6:7]
(179, 285, 310, 351, 652 | 254, 423, 861 | 450, 520)

→ Next a[9] & a[10] are merged, and then a[6:8] & a[9:10]
(179, 285, 310, 351, 652 | 254, 423, 450, 520, 861)

→ At this point there are 2 sorted sub arrays & the final merge produces the fully sorted result.

(179, 254, 285, 310, 351, 423, 450, 520, 652, 861)

- If the time for the merging operations is proportional to 'n', then the computing time for merge sort is described by the recurrence relation.

$$T(n) = \begin{cases} a & n=1, 'a' \text{ a constant} \\ 2T(n/2)+cn & n>1, 'c' \text{ a constant.} \end{cases}$$

→ When 'n' is a power of 2, $n = 2^k$, we can solve this equation by successive substitution.

$$\begin{aligned} T(n) &= 2(2T(n/4) + cn/2) + cn \\ &= 4T(n/4) + 2cn \\ &= 4(2T(n/8) + cn/4) + 2cn \\ &\quad * \\ &\quad * \\ &= 2^k T(1) + kCn. \\ &= an + cn \log n. \end{aligned}$$

→ It is easy to see that if $s^k < n \leq 2^{k+1}$, then $T(n) \leq T(2^{k+1})$. Therefore,
 $T(n) = O(n \log n)$

5. Explain quick sort (UQ:APRIL/MAY'14)

- The divide-and-conquer approach can be used to arrive at an efficient sorting method different from merge sort.
- In merge sort, the file a[1:n] was divided at its midpoint into sub arrays which were independently sorted & later merged.
- In Quick sort, the division into 2 sub arrays is made so that the sorted sub arrays do not need to be merged later.
- This is accomplished by rearranging the elements in a[1:n] such that $a[I] \leq a[j]$ for all I between 1 & n and all j between (m+1) & n for some m, $1 \leq m \leq n$.
- Thus the elements in a[1:m] & a[m+1:n] can be independently sorted.
- No merge is needed. This rearranging is referred to as partitioning.
- Function partition of Algorithm accomplishes an in-place partitioning of the elements of a[m:p-1]

- It is assumed that $a[p] \geq a[m]$ and that $a[m]$ is the partitioning element. If $m=1$ & $p-1=n$, then $a[n+1]$ must be defined and must be greater than or equal to all elements in $a[1:n]$
- The assumption that $a[m]$ is the partition element is merely for convenience, other choices for the partitioning element than the first item in the set are better in practice.
- The function interchange (a, I, j) exchanges $a[I]$ with $a[j]$.

Algorithm: Partition the array $a[m:p-1]$ about $a[m]$

```

1. Algorithm Partition(a,m,p)
2. //within a[m],a[m+1],...,a[p-1] the elements
3. // are rearranged in such a manner that if
4. //initially  $t=a[m]$ , then after completion
5. //  $a[q]=t$  for some  $q$  between  $m$  and
6. //  $p-1$ ,  $a[k] \leq t$  for  $m \leq k < q$ , and
7. //  $a[k] \geq t$  for  $q < k < p$ .  $q$  is returned
8. //Set  $a[p]=\text{infinite}$ .
9. {
10.  $v=a[m]$ ;  $I=m$ ;  $j=p$ ;
11. repeat
12. {
13. repeat
14.    $I=I+1$ ;
15. until( $a[I] \geq v$ );
16. repeat
17.    $j=j-1$ ;
18. until( $a[j] \leq v$ );
19. if ( $I < j$ ) then interchange( $a, I, j$ );
20. }until( $I \geq j$ );
21.  $a[m]=a[j]$ ;  $a[j]=v$ ;
22. return j;
23. }
```

```

1. Algorithm Interchange(a,I,j)
2. //Exchange  $a[I]$  with  $a[j]$ 
3. {
4.  $p=a[I]$ ;
5.  $a[I]=a[j]$ ;
6.  $a[j]=p$ ;
7. }
```

Algorithm: Sorting by Partitioning

```

1. Algorithm Quicksort(p,q)
2. //Sort the elements  $a[p], \dots, a[q]$  which resides
3. //in the global array  $a[1:n]$  into ascending
4. //order;  $a[n+1]$  is considered to be defined
5. // and must be  $\geq$  all the elements in  $a[1:n]$ 
6. {
7. if( $p < q$ ) then // If there are more than one element
8. {
9. // divide  $p$  into 2 subproblems
```

```

10. j=partition(a,p,q+1);
11. //'j' is the position of the partitioning element.
12. //solve the subproblems.
13. quicksort(p,j-1);
14. quicksort(j+1,q);
15. //There is no need for combining solution.
16. }
17. }

```

6. Explain strasson's matrix multiplication (UQ Nov'12& APRIL'12)

- Let A and B be the $2 \times n$ Matrix. The product matrix $C=AB$ is calculated by using the formula,
 $C(i,j) = A(i,k) B(k,j)$ for all 'i' and j between 1 and n.
- The time complexity for the matrix Multiplication is $O(n^3)$.
- Divide and conquer method suggest another way to compute the product of $n \times n$ matrix.
- We assume that N is a power of 2 .In the case N is not a power of 2 ,then enough rows and columns of zero can be added to both A and B .SO that the resulting dimension are the powers of two.
- If $n=2$ then the following formula as a computed using a matrix multiplication operation for the elements of A & B.
- If $n>2$,Then the elements are partitioned into sub matrix $n/2 \times n/2$..since 'n' is a power of 2 these product can be recursively computed using the same formula .This Algorithm will continue applying itself to smaller sub matrix until 'N" become suitable small($n=2$) so that the product is computed directly .
- The formula are

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} * \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$$C_{11}=A_{11}B_{11}+A_{12}B_{21}$$

$$C_{12}=A_{11}B_{12}+A_{12}B_{22}$$

$$C_{21}=A_{21}B_{11}+A_{22}B_{21}$$

$$C_{22}=A_{21}B_{12}+A_{22}B_{22}$$

For EX:

$$4 * 4 = \begin{pmatrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{pmatrix} * \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

The Divide and conquer method

$$\left(\begin{array}{cc|cc} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ \hline 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{array} \right) * \left(\begin{array}{cc|cc} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{array} \right) = \left(\begin{array}{cc|cc} 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 \\ \hline 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 \end{array} \right)$$

- To compute AB using the equation we need to perform 8 multiplication of $n/2 * n/2$ matrix and from 4 addition of $n/2 * n/2$ matrix.
- $C_{i,j}$ are computed using the formula in equation----- $\rightarrow 4$
- As can be sum P, q, R, S, T, U, and V can be computed using 7 Matrix multiplication and 10 addition or subtraction.
- The C_{ij} are required addition 8 addition or subtraction.

$$T(n) = \begin{cases} b & n \leq 2 \text{ a \& b are} \\ 7T(n/2) + an^2 & n > 2 \text{ constant} \end{cases}$$

Finally we get $T(n) = O(n^{\log 27})$

Example

$$\left(\begin{array}{cc} 4 & 4 \\ 4 & 4 \end{array} \right) * \left(\begin{array}{cc} 4 & 4 \\ 4 & 4 \end{array} \right)$$

$$P = (4*4) + (4+4) = 64$$

$$Q = (4+4)4 = 32$$

$$R = 4(4-4) = 0$$

$$S = 4(4-4) = 0$$

$$T = (4+4)4 = 32$$

$$U = (4-4)(4+4) = 0$$

$$V = (4-4)(4+4) = 0$$

$$C_{11} = (64 + 0 - 32 + 0) = 32$$

$$C_{12} = 0 + 32 = 32$$

$$C_{21} = 32 + 0 = 32$$

$$C_{22} = 64 + 0 - 32 + 0 = 32$$

$$\text{So the answer } c(i,j) \text{ is } \begin{vmatrix} 32 & 32 \\ 32 & 32 \end{vmatrix}$$

Since $n/2 * n/2$ matrix can be added in Cn for some constant C, The overall computing time $T(n)$ of the resulting divide and conquer algorithm is given by the sequence.

$$T(n) = \begin{cases} b & n \leq 2 \text{ a \& b are} \\ 8T(n/2) + cn^2 & n > 2 \text{ constant} \end{cases}$$

That is $T(n)=O(n^3)$

* Matrix multiplication are more expensive than the matrix addition $O(n^3)$. We can attempt to reformulate the equation for C_{ij} so as to have fewer multiplication and possibly more addition .

- Strassen has discovered a way to compute the C_{ij} of equation (2) using only 7 multiplication and 18 addition or subtraction.
- Strassen's formula are

$$P = (A_{11} + A_{12})(B_{11} + B_{22})$$

$$Q = (A_{12} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + T$$

$$C_{22} = P + R - Q + V$$

7. Explain greedy method

- Greedy method is the most straightforward designed technique.
- As the name suggest they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decision may have in the future.

DEFINITION:

- A problem with N inputs will have some constraints .any subsets that satisfy these constraints are called a feasible solution.
- A feasible solution that either maximize can minimize a given objectives function is called an optimal solution.

Control algorithm for Greedy Method:

```
1.Algorithm Greedy (a,n)
2.//a[1:n] contain the 'n' inputs
3. {
4.solution =0;//Initialise the solution.
5.For i=1 to n do
6.{
7.x=select(a);
8.if(feasible(solution,x))then
9.solution=union(solution,x);
10.}
11.return solution;
12.}
```

* The function select an input from a[] and removes it. The select input value is assigned to X.

- Feasible is a Boolean value function that determines whether X can be included into the solution vector.
- The function Union combines X with The solution and updates the objective function.
- The function Greedy describes the essential way that a greedy algorithm will once a particular problem is chosen ends the function subset, feasible & union are properly implemented.

Example

- Suppose we have in a country the following coins are available :

Dollars(100 cents)

Quarters(25 cents)

Dimes(10 cents)

Nickel(5 Cents)

Pennies(1 cent)

- Our aim is paying a given amount to a customer using the smallest possible number of coins.
- For example if we must pay 276 cents possible solution then,

→ 1 doll+7 q+ 1 pen→9 coins

→ 2 doll +3Q +1 pen→6 coins

→ 2 doll+7dim+1 nic +1 pen→11 coins.

8. Explain knapsack problem

- We are given n objects and knapsack or bag with capacity M object I has a weight W_i where I varies from 1 to N.
- The problem is we have to fill the bag with the help of N objects and the resulting profit has to be maximum.
- Formally the problem can be stated as

Maximize $\sum x_i p_i$ subject to $\sum x_i W_i \leq M$

Where x_i is the fraction of object and it lies between 0 to 1.

- There are so many ways to solve this problem, which will give many feasible solution for which we have to find the optimal solution.
- But in this algorithm, it will generate only one solution which is going to be feasible as well as optimal.
- First, we find the profit & weight rates of each and every object and sort it according to the descending order of the ratios.
- Select an object with highest p/w ratio and check whether its height is lesser than the capacity of the bag.

- If so place 1 unit of the first object and decrement .the capacity of the bag by the weight of the object you have placed.
- Repeat the above steps until the capacity of the bag becomes less than the weight of the object you have selected .in this case place a fraction of the object and come out of the loop.
- Whenever you selected.

ALGORITHM:

```

1.Algorithm Greedy knapsack (m,n)
2//P[1:n] and the w[1:n]contain the profit
3.// & weight res'.of the n object ordered.
4.//such that  $p[i]/w[i] \geq p[i+1]/w[i+1]$ 
5.//n is the Knapsack size and x[1:n] is the solution vertex.
6.{
7.for I=1 to n do a[I]=0.0;
8.U=n;
9.For I=1 to n do
10.{
11.if (w[i]>u)then break;
13.x[i]=1.0;U=U-w[i]
14.}
15.if(i<=n)then x[i]=U/w[i];
16.}

```

Example:

Capacity=20
 $N=3$, $M=20$
 $W_i=18,15,10$
 $P_i=25,24,15$

$P_i/W_i=25/18=1.36, 24/15=1.6, 15/10=1.5$

Descending Order $\rightarrow P_i/W_i \rightarrow 1.6 \quad 1.5 \quad 1.36$
 $P_i = 24 \quad 15 \quad 25$
 $W_i = 15 \quad 10 \quad 18$
 $X_i = 1 \quad 5/10 \quad 0$

$P_i X_i = 1*24 + 0.5*15 \rightarrow 31.5$

The optimal solution is $\rightarrow 31.5$

X_1	X_2	X_3	$W_i X_i$	$P_i X_i$
$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	16.6	24.25
1	$\frac{2}{5}$	0	20	18.2
0	$\frac{2}{3}$	1	20	31
0	1	$\frac{1}{2}$	20	31.5

Of these feasible solution Solution 4 yield the Max profit .As we shall soon see this solution is optimal for the given problem instance.

9. Explain job scheduling with dead lines

The problem is the number of jobs, their profit and deadlines will be given and we have to find a sequence of job, which will be completed within its deadlines, and it should yield a maximum profit.

Points To remember:

- To complete a job, one has to process the job or a action for one unit of time.
- Only one machine is available for processing jobs.
- A feasible solution for this problem is a subset of j of jobs such that each job in this subset can be completed by this deadline.
- If we select a job at that time ,

→ Since one job can be processed in a single m/c. The other job has to be in its waiting state until the job is completed and the machine becomes free.

→ So the waiting time and the processing time should be less than or equal to the dead line of the job.

ALGORITHM:

Algorithm JS(d, j, n)

//The job are ordered such that $p[1] > p[2] \dots > p[n]$

// $j[i]$ is the i th job in the optimal solution

// Also at terminal $d[J[i]] \leq d[J[i+1]], 1 \leq i < k$

{

$d[0] = J[0] = 0;$

$J[1] = 1;$

$K = 1;$

 For $I = 1$ to n do

 { // consider jobs in non increasing order of $P[I]$; find the position for I and check feasibility insertion

$r = k;$

 while($(d[J[r]] > d[i])$ and

$(d[J[r]] \neq r)$) do $r = r - 1;$

 if ($(d[J[r]] < d[I])$ and $(d[I] > r)$) then

 {

 for $q = k$ to $(r + 1)$ step -1 do $J[q + 1] = j[q]$

$J[r + 1] = i;$

$K = k + 1;$

 }

 }

 return $k;$

}

Example :

1. $n = 5$ ($P_1, P_2, \dots, P_5$) = (20, 15, 10, 5, 1)

 ($d_1, d_2, \dots, d_5$) = (2, 2, 1, 3, 3)

<i>Feasible solution</i>	<i>Processing Sequence</i>	<i>Value</i>
(1)	(1)	20
(2)	(2)	15
(3)	(3)	10
(4)	(4)	5
(5)	(5)	1
(1,2)	(2,1)	35
(1,3)	(3,1)	30
(1,4)	(1,4)	25
(1,5)	(1,5)	21
(2,3)	(3,2)	25
(2,4)	(2,4)	20
(2,5)	(2,5)	16
(1,2,3)	(3,2,1)	45
(1,2,4)	(1,2,4)	40

The Solution 13 is optimal

2. $n=4$ $(P1,P2,...P4)=(100,10,15,27)$
 $(d1,d2....d4)=(2,1,2,1)$

<i>Feasible solution</i>	<i>Processing Sequence</i>	<i>Value</i>
(1,2)	(2,1)	110
(1,3)	(1,3)	115
(1,4)	(4,1)	127
(2,3)	(9,3)	25
(2,4)	(4,2)	37
(3,4)	(4,3)	42
(1)	(1)	100
(2)	(2)	10
(3)	(3)	15
(4)	(4)	27

The solution 3 is optimal.

10. Explain minimum spanning tree (UQ Nov'12, April/May'14)

- Let $G(V,E)$ be an undirected connected graph with vertices 'v' and edge 'E'.
- A sub-graph $t=(V,E')$ of the G is a Spanning tree of G iff 't' is a tree.³
- The problem is to generate a graph $G'=(V,E)$ where 'E' is the subset of E,G' is a Minimum spanning tree.
- Each and every edge will contain the given non-negative length .connect all the nodes with edge present in set E' and weight has to be minimum.

NOTE:

- We have to visit all the nodes.

- The subset tree (i.e) any connected graph with 'N' vertices must have at least N-1 edges and also it does not form a cycle.

Definition:

- A spanning tree of a graph is an undirected tree consisting of only those edge that are necessary to connect all the vertices in the original graph.
- A Spanning tree has a property that for any pair of vertices there exist only one path between them and the insertion of an edge to a spanning tree form a unique cycle.

Application of the spanning tree:

1. Analysis of electrical circuit.
2. Shortest route problems.

Minimum cost spanning tree:

- The cost of a spanning tree is the sum of cost of the edges in that trees.
- There are 2 method to determine a minimum cost spanning tree are

1. Kruskal's Algorithm
2. Prim's Algorithm.

KRUSKAL'S ALGORITHM:

In kruskal's algorithm the selection function chooses edges in increasing order of length without worrying too much about their connection to previously chosen edges, except that never to form a cycle. The result is a forest of trees that grows until all the trees in a forest (all the components) merge in a single tree.

- In this algorithm, a minimum cost-spanning tree 'T' is built edge by edge.
- Edge are considered for inclusion in 'T' in increasing order of their cost.
- An edge is included in 'T' if it doesn't form a cycle with edge already in T.
- To find the minimum cost spanning tree the edge are inserted to tree in increasing order of their cost

Algorithm:

```

Algorithm kruskal(E, cost, n, t)
//E → set of edges in G has 'n' vertices.
//cost[u, v] → cost of edge (u, v). t → set of edge in minimum cost spanning tree
// the first cost is returned.
{
for i=1 to n do parent[i]=-1;
I=0; mincost=0.0;
While((I<n-1) and (heap not empty)) do
{
j=find(u);
k=find(v);
if(j not equal k) then
{
i=i+1
t[i,1]=u;
t[i,2]=v;
mincost=mincost+cost[u,v];
}
}
}

```

```

union(j,k);
}
}
if(i not equal n-1) then write("No spanning tree")
else return minimum cost;
}

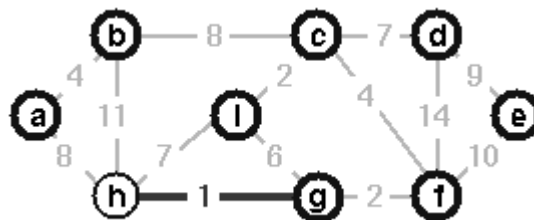
```

Analysis

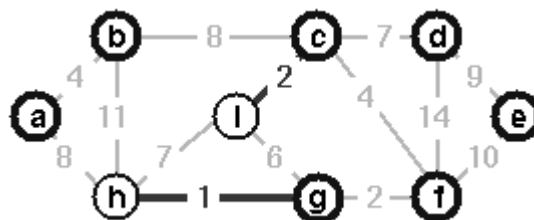
- The time complexity of minimum cost spanning tree algorithm in worst case is $O(|E|\log|E|)$,
 → where E is the edge set of G.

Example: Step by Step operation of Kruskal algorithm.

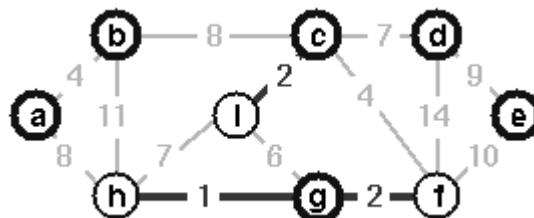
Step 1. In the graph, the Edge(g, h) is shortest. Either vertex g or vertex h could be representative. Lets choose vertex g arbitrarily.



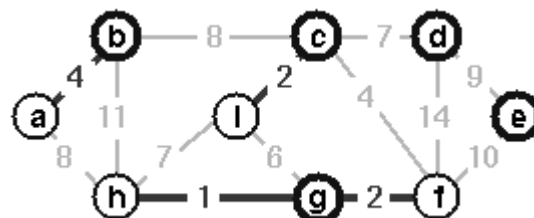
Step 2. The edge (c, i) creates the second tree. Choose vertex c as representative for second tree.



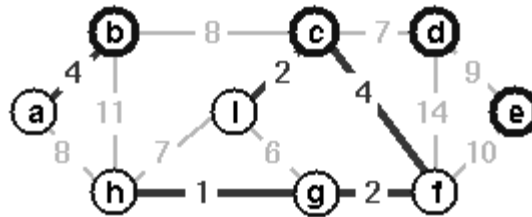
Step 3. Edge (g, g) is the next shortest edge. Add this edge and choose vertex g as representative.



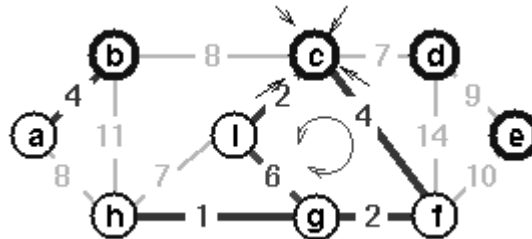
Step 4. Edge (a, b) creates a third tree.



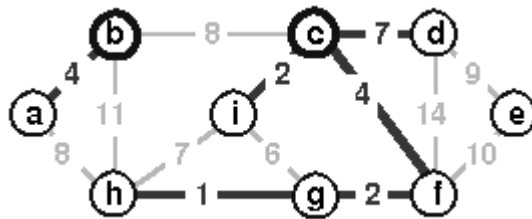
Step 5. Add edge (c, f) and merge two trees. Vertex c is chosen as the representative.



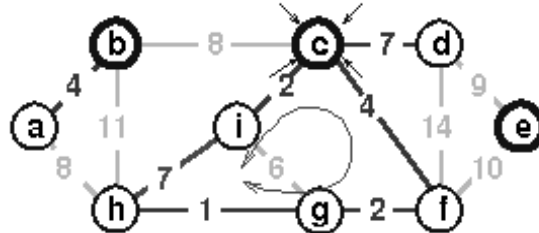
Step 6. Edge (g, i) is the next next cheapest, but if we add this edge a cycle would be created. Vertex c is the representative of both.



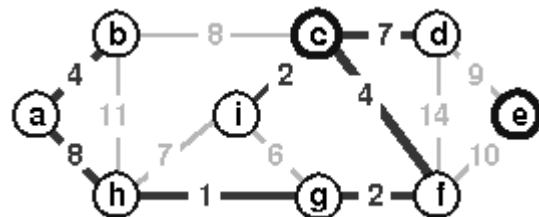
Step 7. Instead, add edge (c, d).



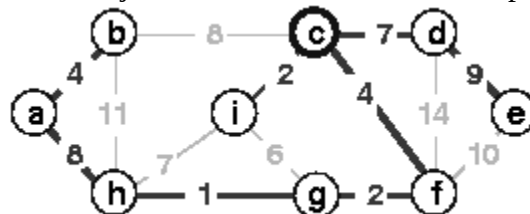
Step 8. If we add edge (h, i), edge(h, i) would make a cycle.



Step 9. Instead of adding edge (h, i) add edge (a, h).

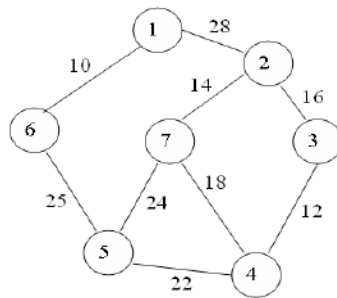


Step 10. Again, if we add edge (b, c), it would create a cycle. Add edge (d, e) instead to complete the spanning tree. In this spanning tree all trees joined and vertex c is a sole representative.



PRIM'S ALGORITHM

Start from an arbitrary vertex (root). At each stage, add a new branch (edge) to the tree already constructed; the algorithm halts when all the vertices in the graph have been reached.

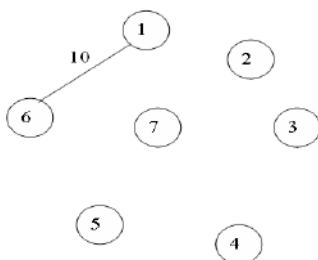


Algorithm prims(e, cost, n, t)

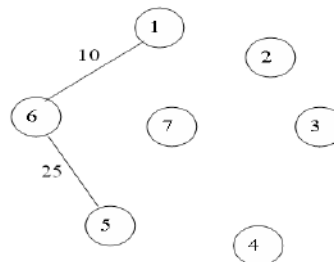
```
{
  Let (k,l) be an edge of minimum cost in E;
  Mincost :=cost[k,l];
  T[1,1]:=k; t[1,2]:=l;
  For I:=1 to n do
    If (cost[i,l]<cost[i,k]) then near[i]:=l;
    Else near[i]:=k;
  Near[k]:=near[l]:=0;
  For i:=2 to n-1 do
    {
      Let j be an index such that near[j]≠0 and
      Cost[j,near[j]] is minimum;
      T[i,1]:=j; t[i,2]:=near[j];
      Mincost:=mincost+ Cost[j,near[j]];
      Near[j]:=0;
      For k:=0 to n do
        If near((near[k]≠0) and (Cost[k,near[k]]>cost[k,j])) then
          Near[k]:=j;
    }
  Return mincost;
}
```

- The prims algorithm will start with a tree that includes only a minimum cost edge of G.
- Then, edges are added to the tree one by one. the next edge (i,j) to be added in such that I is a vertex included in the tree, j is a vertex not yet included, and cost of (i,j), cost[i,j] is minimum among all the edges.
- The working of prims will be explained by following diagram

Step 1:

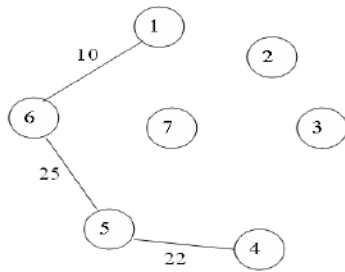


Step 2:

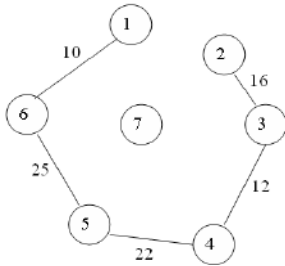


Step 3:

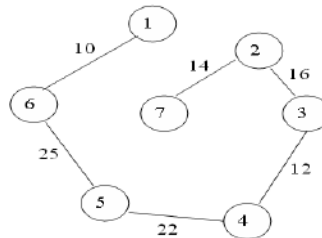
Step 4:



Step 5:



Step 6:



11. Explain Single-source shortest path

Graphs can be used to represent the highway structure of a state or country with vertices representing cities and edges representing sections of highway. The edges can then be assigned weights which may be either the distance between the two cities connected by the edge or the average time to drive along that section of highway. A motorist wishing to drive from city A to B would be interested in answers to the following questions:

1. Is there a path from A to B?
2. If there is more than one path from A to B? Which is the shortest path?

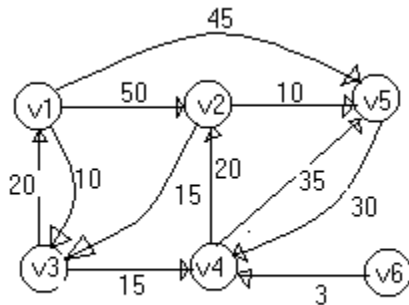


Fig 7.1

The problems defined by these questions are special case of the path problem we study in this section. The length of a path is now defined to be the sum of the weights of the edges on that path. The starting vertex of the path is referred to as the source and the last vertex the destination. The graphs are digraphs representing streets. Consider a digraph $G=(V,E)$, with the distance to be traveled as weights on the edges. The problem is to determine the shortest path from v_0 to all the remaining vertices of G . It is assumed that all the weights associated with the edges are positive. The shortest path between v_0 and some other node v is an ordering among a subset of the edges. Hence this problem fits the ordering paradigm.

Example:

Consider the digraph of fig 7-1. Let the numbers on the edges be the costs of travelling along that route. If a person is interested travel from v_1 to v_2 , then he encounters many paths. Some of them are

1. $v1 \rightarrow v2 = 50$ units
2. $v1 \rightarrow v3 \rightarrow v4 \rightarrow v2 = 10+15+20=45$ units
3. $v1 \rightarrow v5 \rightarrow v4 \rightarrow v2 = 45+30+20= 95$ units
4. $v1 \rightarrow v3 \rightarrow v4 \rightarrow v5 \rightarrow v4 \rightarrow v2 = 10+15+35+30+20=110$ units

The cheapest path among these is the path along $v1 \rightarrow v3 \rightarrow v4 \rightarrow v2$. The cost of the path is $10+15+20 = 45$ units. Even though there are three edges on this path, it is cheaper than travelling along the path connecting $v1$ and $v2$ directly i.e., the path $v1 \rightarrow v2$ that costs 50 units. One can also notice that, it is not possible to travel to $v6$ from any other node.

To formulate a greedy based algorithm to generate the cheapest paths, we must conceive a multistage solution to the problem and also of an optimization measure. One possibility is to build the shortest paths one by one. As an optimization measure we can use the sum of the lengths of all paths so far generated. For this measure to be minimized, each individual path must be of minimum length. If we have already constructed i shortest paths, then using this optimization measure, the next path to be constructed should be the next shortest minimum length path. The greedy way to generate these paths in non-decreasing order of path length. First, a shortest path to the nearest vertex is generated. Then a shortest path to the second nearest vertex is generated, and so on.

A much simpler method would be to solve it using matrix representation. The steps that should be followed is as follows,

Step 1: find the adjacency matrix for the given graph. The adjacency matrix for fig 7.1 is given below

	V1	V2	V3	V4	V5	V6
V1	-	50	10	Inf	45	Inf
V2	Inf	-	15	Inf	10	Inf
V3	20	Inf	-	15	inf	Inf
V4	Inf	20	Inf	-	35	Inf
V5	Inf	Inf	Inf	30	-	Inf
V6	Inf	Inf	Inf	3	Inf	-

Step 2: consider $v1$ to be the source and choose the minimum entry in the row $v1$. In the above table the minimum in row $v1$ is 10.

Step 3: find out the column in which the minimum is present, for the above example it is column $v3$. Hence, this is the node that has to be next visited.

Step 4: compute a matrix by eliminating $v1$ and $v3$ columns. Initially retain only row $v1$. The second row is computed by adding 10 to all values of row $v3$.

The resulting matrix is

	V2	V4	V5	V6
V1 $\square$ Vw	50	Inf	45	Inf
V1 $\square$ V3 $\square$ Vw	10+inf	10+15	10+inf	10+inf
Minimum	50	25	45	inf

Step 5: find the minimum in each column. Now select the minimum from the resulting row. In the above example the minimum is 25. Repeat step 3 followed by step 4 till all vertices are covered or single column is left.

The solution for the fig 7.1 can be continued as follows

	V2	V5	V6
V1 $\square$ Vw	50	45	Inf
V1 $\square$ V3 $\square$ V4 $\square$ Vw	25+20	25+35	25+inf
Minimum	45	45	inf

	V5	V6
V1 $\square$ Vw	45	Inf
V1 $\square$ V3 $\square$ V4 $\square$ V2 $\square$ Vw	45+10	45+inf
Minimum	45	inf

	V6
V1 $\square$ Vw	Inf
V1 $\square$ V3 $\square$ V4 $\square$ V2 $\square$ V5 $\square$ Vw	45+inf
Minimum	inf

Finally the cheapest path from v1 to all other vertices is given by V1, V3,V4, V2, V5.

Dijkstra's algorithm:

Procedure Dijkstra

{ Dijkstra computes the cost of the shortest path from vertex 1 to every vertex of a directed graph }
begin

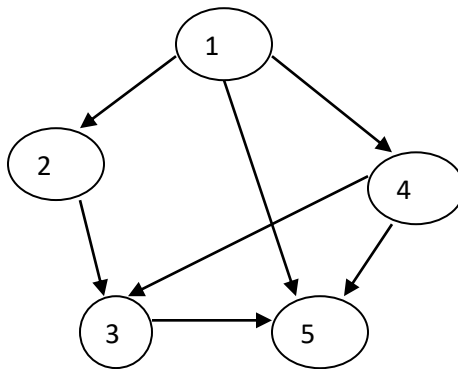
1. $s = \{1\};$
2. for $i := 2$ to n do
3. $D[i] := c[1,i];$ {initialize D}
4. for $i = 1$ to $n-1$ do begin

```

5.         choose a vertex w in V-S such that D[w] is minimum
6.         Add w to S
7.     for each vertex v in V-S do
8.          $D[v] := \min(D[v], D[w] + c[w, v]);$ 
9.     end
end{Dijkstra}

```

Example 2:



Cost adjacency matrix:

	1	2	3	4	5
1	9999	10	9999	30	100
2	9999	9999	50	9999	9999
3	9999	9999	9999	9999	10
4	9999	9999	20	9999	60
5	9999	9999	9999	9999	9999

$S = \{1\}$

$V = \{1, 2, 3, 4, 5\}$

For $i=2$ to 5 do

$i=2$ $D[2]=10$

$i=3$ $D[3]=9999$

$i=4$ $D[4]=30$

$i=5$ $D[5]=100$

for $i=1$ to 4 do

$D[3]=60$

$D[5]=30$

$i=1$

$V-S = \{2, 3, 4, 5\}$

$D[w]=10;$

$w=2(\text{minimum})$

$S = \{1, 2\}$

For each vertex v in $V-S$ i.e) $V-S = \{3, 4, 5\} \Rightarrow$ for $v=3, 4, 5$

When $v=3$

$D[3] = \min(D[3], D[2] + c[2, 3])$

$D[3] = \min(9999, 10 + 50)$

$D[3] = 60$

When $v=4$

$D[4] = \min(D[4], D[2] + c[2,4])$

$D[4] = \min(30, 10 + 9999)$

$D[4] = 30$

When $v=5$

$D[5] = \min(D[5], D[2] + c[2,5])$

$D[5] = \min(100, 10 + 9999)$

$D[5] = 100$

$D[2] = 10$	$D[3] = 60$	$D[4] = 30$	$D[5] = 100$
-------------	-------------	-------------	--------------

$i=2$

$V-S = \{3,4,5\}$

$D[w] = 30$; $w=4$ (minimum)

$S = \{1,2,4\}$

For each vertex v in $V-S \setminus \{3,5\} \Rightarrow$ for $v=3,5$

When $v=3$

$D[3] = \min(D[3], D[4] + c[4,3])$

$D[3] = \min(60, 30 + 20)$

$D[3] = 50$

When $v=5$

$D[5] = \min(D[5], D[4] + c[4,5])$

$D[5] = \min(100, 30 + 60)$

$D[5] = 90$

$D[3] = 50$	$D[5] = 90$
-------------	-------------

$i=3$

$V-S = \{3,5\}$

$D[w] = 50$; $w=3$ (minimum)

$S = \{1,2,4,3\}$

For each vertex v in $V-S \setminus \{5\} \Rightarrow$ for $v=5$

When $v=5$

$D[5] = \min(D[5], D[3] + c[3,5])$

$D[5] = \min(90, 50 + 10)$

$D[5] = 60$

$D[5] = 60$

$i=4$

$V-S = \{5\}$

$D[w] = 60$; $w=5$ (minimum)

$S = \{1,2,4,3,5\}$

Analysis:

Run time is $O((n + |E|) \log n)$.

12. Explain optimal storage on tapes (UQ APRIL'12)

- We have n no of programs. The length of the tape l & length of each program l_i , $1 \leq i \leq n$.
- The objective of the problem is to store ' n ' no of programs on a tape with the constraint: Sum of the length of all the programs should not exceed the length of the tape l . (i.e) All the programs can be stored on the tape iff the sum of the length of the program is atmost l .
- Assume whenever a program is retrieved from the tape, tape is initially positioned at the front.

- Time needed (t_j) to retrieve the program l_j is proportional to $\sum l_i^k$ where $k=1$ to j .
- Program P_1 having length l_1 takes t_1 time to store it.
- Program P_2 having length l_2 takes t_2 time to store it.
- Program P_3 having length l_3 takes t_3 time to store it.
- We have to store l_1, l_2, l_3 in l_j ;
- If all the programs are equally retrieved, then
Mean retrieval time $d = (1/n) \sum t_j, 1 \leq j \leq n$
 $d = (t_1 + t_2 + \dots + t_n)/n$
- Our aim is to minimize the mean retrieval time. To minimize MRT we have to find the permutation for n programs. (i.e) We must consider the order in which programs are saved.

Example:

$P_1 \rightarrow l_1 \rightarrow 5$

$P_2 \rightarrow l_2 \rightarrow 10$

$P_3 \rightarrow l_3 \rightarrow 3$

Possible orderings = $n! = 3! = 6$

I	d(i)	MRT
1, 2, 3	$5+15+18$	38
1, 3, 2	$5+8+18$	31
2, 3, 1	$10+13+18$	41
2, 1, 3	$10+15+18$	43
3, 1, 2	$3+8+18$	29(min)
3, 2, 1	$3+13+18$	34

The 5th order 3,2,1 takes minimum MRT 29. So that if the programs are stored in the sequence 3,2,1 then it is an optimal solution.

Analysis:

The greedy method simply requires to store the programs in non decreasing order of their lengths. This ordering can be carried out in $O(n \log n)$ time using an efficient sorting algorithm (heap sort).

13. Explain optical merge patterns

- We have 2 sorted files containing n and m records. The file f_1 contains n records. The file f_2 contains m records.
- These 2 files could be merged together and resultant file is in also sorted order.
- Time taken for merging these two files is $O(m+n)$.
- If we have more than 2 files, then we can merge the files in some order.
- Let us take 4 files say x_1, x_2, x_3 & x_4 . We can merge these 4 files in any order.
- For example:
 1. Merge x_1 & $x_2 \Rightarrow y_1$
Merge y_1 & $x_3 \Rightarrow y_2$
Merge y_2 & $x_4 \Rightarrow y_3$
 2. Merge x_1 & $x_2 \Rightarrow y_1$
Merge x_3 & $x_4 \Rightarrow y_2$
Merge y_1 & $y_2 \Rightarrow y_3$
- Given n sorted files. We have to sort the n sorted files & store it in a single file in sorted order.
- We can merge the files in any order, but it should take minimum amount of computing time.

Example 1:

$n=3$

File x_1 length=30

File x2 length=20

File x3 length=10

Way 1:

Merge x1 & x2 - 50 record moves

Merge(x1 & x2)&x3 - 60 record moves

Total no of record moves -110 record moves

Way 2:

Merge x2 & x3 - 30 record moves

Merge(x2 & x3)&x1 - 60 record moves

Total no of record moves - 90 record moves

Minimum record moves is in way2(90).So the second merge pattern is faster than first.

By greedy method, optimal merge pattern suggests:At each step merge the 2 smallest files together.

Example 2 :

x1	x2	x3	x4	x5
20	30	10	5	30

Merge x3+x4 $\rightarrow 10+5=15 \Rightarrow z1$

Merge z1+x1 $\rightarrow 15+20=35 \Rightarrow z2$

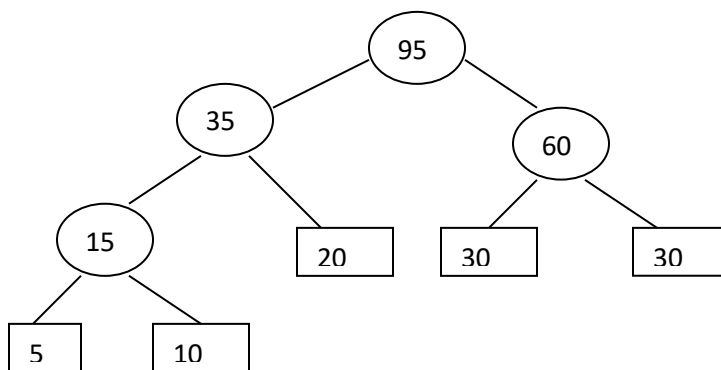
Merge x2+x5 $\rightarrow 30+30=60 \Rightarrow z3$

Merge z2+z3 $\rightarrow 35+60=95$

Total no. of record moves = 205moves

2 Way Merge Pattern:

- 1.It is represented by binary merge trees.
- 2.Leaf nodes are drawn as squares.
- 3.Nodes are called as external nodes.
- 4.Remaining nodes are drawn as circles.(ie) internal nodes
5. Each internal node has exactly 2 children. Internal node represents the file obtained by its 2 children.



In the above tree, square represents a single sorted file and circle represents the merging of 2 files.

Pondicherry University Questions

2 Marks

1. Define the divide and conquer method. (UQ APRIL '13 & APRIL '12) (Ref.Pg.No.1 Qn.No.1)
2. What is the binary search? (UQ APRIL '12) (Ref.Pg.No.1 Qn.No.5)
3. What is the Quick sort? (UQ Nov'10& APRIL '12) (Ref.Pg.No.2 Qn.No.8)
4. Is insertion sort better than the merge sort? (UQ APRIL '13) (Ref.Pg.No.2 Qn.No.10)
5. What is Knapsack problem? (UQ Nov'12) (Ref.Pg.No.3 Qn.No.17)
6. Define minimum spanning tree (UQ Nov'12) (Ref.Pg.No.4 Qn.No.26)
7. What is greedy method? (UQ:Apr/May'14) (Ref.Pg.No.3 Qn.No.19)

11 Marks

1. Write algorithm and explain binary search (UQ APRIL '13 & APRIL '12/May'16)(Ref.Pg.No.6 Qn.No.2)
2. Explain Maximum and Minimum (UQ APRIL '13/ Nov'16) (Ref.Pg.No.8 Qn.No.3)
3. Explain merge sort (UQ APRIL '12,**APRIL/MAY'14**) (Ref.Pg.No.10 Qn.No.4)
4. Explain strasson's matrix multiplication (UQ Nov'12& APRIL '12) (Ref.Pg.No.15 Qn.No.6)
5. Explain minimum spanning tree (UQ Nov'12, Apr/May'14/Nov'16/ May'14) (Ref.Pg.No.21 Qn.No.10)
6. Explain optimal storage on tapes (UQ APRIL '12) (Ref.Pg.No.30 Qn.No.12)
7. Explain quick sort (UQ:APRIL/MAY'14) (Ref.Pg.No.13 Qn.No.5)